Assignment 4: Yoonsang You & Anthony

In this phase, we conducted a manual code review to identify areas for improvement and refactored eight different code smells. Each smell was analyzed, and refactorings were applied to enhance readability, maintainability, and overall design quality.

1. **Exception Swallowing - *TileManager.java***

   The TileManager file contains a *try catch* statement where there exists an x exception swallowing where all exceptions are silently dealt. This may hide potential exceptions that may arise when coding but it makes it difficult to debug.

   Solution: Give visibility in the exception and log it.

   Committed

```
} catch (Exception e) {
    e.printStackTrace();
    throw new RuntimeException(message:"Error loading map data", e);
}
```

2. **Large Method - *TileManager.java***

   Within the TileManager the method *DrawBottom* manages too many drawing roles.

   Solution: Break the method down to smaller chunks by using helper function

   Committed

```
private void drawBottom(Graphics2D g2) {
    drawWallTiles(g2, Layer.BOTTOM);
    drawEntityShadows(g2);
}

private void drawWallTiles(Graphics2D g2, Layer layer) {
    for (Tile wall : getScreenTiles()) {
        if (wall.layer == layer) wall.draw(g2);
    }
}

private void drawEntityShadows(Graphics2D g2) {
    for (Enemy enemy : gp.getEnemyGenerator().getEnemies()) {
        if (enemy instanceof Camera) ((Camera) enemy).getSpotlight().draw(g2);
        enemy.drawShadow(g2);
    }
    for (Reward reward : gp.getRewardGenerator().getRewards()) {
        reward.drawShadow(g2);
    }
    gp.getPlayer().drawShadow(g2);
}
```

3. **Hard Coded - *Camera.java***

   Within the *Camera.java* values such as range, spotlightTimer are hard coded.

   Solution: Change the values into constants.

   Committed

```
private static final int DEFAULT_DETECTION_RANGE = 96;
private static final double SPOTLIGHT_TIMER_DURATION = 1.25;
private static final int SPOTLIGHT_WIDTH = 128;
private static final int SPOTLIGHT_HEIGHT = 72;
private static final int SPOTLIGHT_OFFSET_X = 32;
private static final int SPOTLIGHT_OFFSET_Y = 16;
private static final int CAMERA_SPEED = 3;

private Entity spotlight;
private Timer spotlightTimer = new Timer(SPOTLIGHT_TIMER_DURATION);
private boolean spotlightOn = false;
```

4. **Redundant Code - *Level.java***

Within the *loadlevel()* method there is a redundant line of code: reader.close() that is not necessary because the try with resources already does it.
Solution: Delete the redundant code
Committed

```
// reader.close();
```

5. **Detailed Exception handling - *Level.java***
   Exceptions must be descriptive to simplify the debugging process. In the loadLevel() method, we refactored the catch statement such that it now handles 3 different exceptions when loading the map.

```
} catch (IOException e) {
    System.err.print("Failed to Load Map at " + path);
    e.printStackTrace();
    //added more exceptions to catch
} catch (NumberFormatException e) {
    System.err.println("Invalid number format at " + path);
    e.printStackTrace();
    //added more exceptions to catch
} catch (Exception e) {
    System.err.println("An unexpected error occurred while loading level: " + path);
    e.printStackTrace();
}
```

6. **Long Method - *Dog.java***
   The wander method handled movement logic, collision handling and animation. We refactored it such that it now has a single use method to disperse the work.

```
private void moveTowards(Vector delta) {
    if (delta.x > 1) move(Direction.RIGHT);
    else if (delta.x < -1) move(Direction.LEFT);
    if (delta.y > 1) move(Direction.DOWN);
    else if (delta.y < -1) move(Direction.UP);
}
```

7. **Key Pressed and Released Handling**

```java
/**
 * Updates the movement flags when a key is pressed. Sets the corresponding flag to true
 * if the pressed key is 'W', 'A', 'S', or 'D'.
 *
 * @param e the key event triggered when a key is pressed
 */
@Override  Yoon +1
public void keyPressed(KeyEvent e) {
    int key = e.getKeyCode();
    if (key == KeyEvent.VK_UP) up = true;
    if (key == KeyEvent.VK_LEFT) left = true;
    if (key == KeyEvent.VK_DOWN) down = true;
    if (key == KeyEvent.VK_RIGHT) right = true;

    if (key == KeyEvent.VK_W) up = true;
    if (key == KeyEvent.VK_A) left = true;
    if (key == KeyEvent.VK_S) down = true;
    if (key == KeyEvent.VK_D) right = true;
}

/**
 * Updates the movement flags when a key is released. Sets the corresponding flag to false
 * if the released key is 'up', 'left', 'down', or 'right'.
 *
 * @param e the key event triggered when a key is released
 */
@Override  Yoon +1
public void keyReleased(KeyEvent e) {
    int key = e.getKeyCode();
    if (key == KeyEvent.VK_UP) up = false;
    if (key == KeyEvent.VK_LEFT) left = false;
    if (key == KeyEvent.VK_DOWN) down = false;
    if (key == KeyEvent.VK_RIGHT) right = false;

    if (key == KeyEvent.VK_W) up = false;
    if (key == KeyEvent.VK_A) left = false;
    if (key == KeyEvent.VK_S) down = false;
    if (key == KeyEvent.VK_D) right = false;
}
```

Code Smell: Duplication, Inefficient Conditionals

In the provided keyPressed() and keyReleased() methods, there is significant duplication in the logic for checking key presses. The logic for setting the flags for movement directions (up, down, left, right) is repeated for both KeyEvent.VK_UP/KeyEvent.VK_W and similarly for other directions. This is inefficient and leads to duplicated code that could be more maintainable.

Refactoring:

For a more scalable approach, we could use maps and key flags, where we could map a key with its corresponding flag. This would allow us to add or remove keys without changing the logic too much. Another approach is to use a switch statement. The refactored version eliminates code duplication by grouping similar key checks together, reducing redundancy. Here's the improved version:

Benefits of Refactoring:

- **Code Duplication:** The refactored code eliminates redundant if statements and consolidates key checks.
- **Readability:** The refactored code is more concise, and the logic is easier to follow.

8. Player Update Method

```java
/**
 * Updates the player's state, including position based on input, collision detection, and animation frame updates.
 * Handles movement along both x and y axes, resetting position if collisions are detected.
 */
@Override  VivanPrasad +3
public void update() {
    // Handle movement input from the key detector
    if (keyh.up || keyh.left || keyh.down || keyh.right) {
        if (keyh.left) move(Direction.LEFT);
        else if (keyh.right) move(Direction.RIGHT);
        if (keyh.up) move(Direction.UP);
        else if (keyh.down) move(Direction.DOWN);

        // Footstep sound effect
        int oldFrame = getAnimationPlayer().getFrame();
        super.update();
        int newFrame = getAnimationPlayer().getFrame();
        if (oldFrame != newFrame && newFrame % 2 == 0) gp.getSFX().play( sfxName: "footstep");
        getAnimationPlayer().playAnimation(direction.name());

    } else idle();
    updateCamera();
}
```

Code Smell: Unnecessary branching, logic complexity, Inefficient Conditional Logic and Potential Redundancy

In the update() method, the movement logic for handling the player's movement based on key presses uses both if and else if statements. This approach could be improved by simplifying the movement logic to reduce complexity.

Refactoring:

By restructuring the conditional checks, the refactored code avoids redundant conditions and improves clarity by removing unnecessary else if branches. Here's how the method can be refactored:

```java
@Override  VivanPrasad +3
public void update() {
    // Handle movement input from the key detector
    if (keyh.up || keyh.left || keyh.down || keyh.right) {
        if (keyh.left) move(Direction.LEFT);
        if (keyh.right) move(Direction.RIGHT);
        if (keyh.up) move(Direction.UP);
        if (keyh.down) move(Direction.DOWN);

        // Footstep sound effect
        int oldFrame = getAnimationPlayer().getFrame();
        super.update();
        int newFrame = getAnimationPlayer().getFrame();
        if (oldFrame != newFrame && newFrame % 2 == 0) gp.getSFX().play( sfxName: "footstep");
        getAnimationPlayer().playAnimation(direction.name());

    } else idle();
    updateCamera();
}
```

Benefits of Refactoring:

- **Simplified Conditions:** The use of if statements for movement makes the code clearer by removing unnecessary else if conditions.
- **Separation of Logic:** Movement logic and animation frame updates are clearly separated

9. **Level Loading Method**

```java
/**
 * Method to Load the map
 */
private void loadLevel() {  1 usage  Yonny04 +1
    String path = String.format("/level/%d.level", levelNumber);

    try (BufferedReader reader = ResourceLoader.loadFile(path)){
        // Valuable Count
        VALUABLES_COUNT = Integer.parseInt(reader.readLine());

        //Time Limit
        TIME_LIMIT = Double.parseDouble(reader.readLine());

        // For Reading Enemy Count
        String[] enemiesString = reader.readLine().split( regex: " ");

        GUARDS_COUNT = Integer.parseInt(enemiesString[0]);
        DOGS_COUNT = Integer.parseInt(enemiesString[1]);
        CAMERA_COUNT = Integer.parseInt(enemiesString[2]);
        LASER_COUNT = Integer.parseInt(enemiesString[3]);

        String[] mapString = reader.readLine().split( regex: " ");
        Vector mapSize = new Vector(Integer.parseInt(mapString[0]), Integer.parseInt(mapString[1]));
        gp.getTileManager().loadMap(reader, mapSize.x,mapSize.y);
        reader.close();

    } catch (IOException e) {
        System.err.print("Failed to Load Map at " + path);
        e.printStackTrace();
    } catch (NumberFormatException e) {
        System.err.println("Invalid number format at " + path);
        e.printStackTrace();
    } catch (Exception e) {
        System.err.println("An unexpected error occurred while loading level: " + path);
        e.printStackTrace();
    }
}
```

Code Smell: Long Method and High Coupling

The loadLevel() method handles multiple tasks: reading the level data, parsing it, and interacting with the TileManager. This results in a long method that does too much. Additionally, the method directly references the gp.getTileManager() object, creating tight coupling and making the code harder to test and maintain.

Refactoring:

By refactoring the loadLevel() method into smaller methods, each with a single responsibility, we improve readability and maintainability. We also address the high coupling by isolating the interaction with the TileManager. Here's the refactored version:

```
/**
 * Method to Load the map
 */
private void loadLevel() {  1 usage   VivanPrasad +1 *
    String path = String.format("/level/%d.level", levelNumber);
    try (BufferedReader reader = ResourceLoader.loadFile(path)) {
        parseLevelData(reader);
        loadMap(reader);
    } catch (IOException | NumberFormatException e) {
        handleLoadingError(e, path);
    } catch (Exception e) {
        handleUnexpectedError(e, path);
    }
}
```

```
private void parseLevelData(BufferedReader reader) throws IOException, NumberFormatException {  1 usage   new *
    // Parsing valuable count
    VALUABLES_COUNT = Integer.parseInt(reader.readLine());

    // Parsing time limit
    TIME_LIMIT = Double.parseDouble(reader.readLine());

    // Parsing enemy counts
    String[] enemiesString = reader.readLine().split( regex: " ");
    GUARDS_COUNT = Integer.parseInt(enemiesString[0]);
    DOGS_COUNT = Integer.parseInt(enemiesString[1]);
    CAMERA_COUNT = Integer.parseInt(enemiesString[2]);
    LASER_COUNT = Integer.parseInt(enemiesString[3]);
}

private void loadMap(BufferedReader reader) throws IOException {  1 usage   new *
    String[] mapString = reader.readLine().split( regex: " ");
    Vector mapSize = new Vector(Integer.parseInt(mapString[0]), Integer.parseInt(mapString[1]));
    gp.getTileManager().loadMap(reader, mapSize.x, mapSize.y);
}
```

```
private void handleLoadingError(Exception e, String path) {  1 usage   new *
    System.err.println("Error loading level at " + path);
    e.printStackTrace();
}

private void handleUnexpectedError(Exception e, String path) {  1 usage   new *
    System.err.println("An unexpected error occurred while loading level: " + path);
    e.printStackTrace();
}
```